

C++ Project Setup Guide - VS Code + MinGW

Table of Contents

1. [Prerequisites](#)
2. [Project Structure](#)
3. [Create Project Files](#)
4. [VS Code Configuration](#)
5. [Build & Run](#)
6. [Debug](#)
7. [Troubleshooting](#)

Prerequisites

Install MinGW (g++ compiler)

1. Download MinGW from <https://www.mingw-w64.org/>
2. Install to `c:\MinGW`
3. Add to system PATH:
 - Open **System Properties** → **Environment Variables**
 - Edit **Path** → Add `c:\MinGW\bin`

Verify Installation

Open Command Prompt and run:

```
g++ --version  
make --version
```

Install VS Code Extensions

```
ms-vscode.cpptools          # C/C++ IntelliSense
```

Press `Ctrl+Shift+X` → Search and install above extensions.

Project Structure

```
ProjectName/  
├─ .vscode/  
│   ├── tasks.json  
│   ├── launch.json  
│   └─ c_cpp_properties.json  
├─ inc/          # Header files (.h, .hpp)  
├─ src/          # Source files (.cpp)  
├─ obj/          # Object files (.o)  
├─ bin/          # Executables (.exe)  
├─ lib/          # Libraries  
└─ Makefile
```

Create Folders

```
# Run in PowerShell  
New-Item -ItemType Directory -Path "inc","src","obj","bin","lib",".vscode" -Force
```

Create Project Files

1. Header File: inc/calculator.h

```
#ifndef CALCULATOR_H
#define CALCULATOR_H

class Calculator {
public:
    Calculator();
    ~Calculator();
    double add(double a, double b);
    double subtract(double a, double b);
    double multiply(double a, double b);
    double divide(double a, double b);
};

#endif
```

2. Implementation: src/calculator.cpp

```
#include <calculator.h>          // Angular brackets work via -I inc

Calculator::Calculator() {}
Calculator::~Calculator() {}

double Calculator::add(double a, double b) {
    return a + b;
}

double Calculator::subtract(double a, double b) {
    return a - b;
}

double Calculator::multiply(double a, double b) {
    return a * b;
}

double Calculator::divide(double a, double b) {
    if (b == 0) return 0;
    return a / b;
}
```

3. Main File: src/main.cpp

```
#include <iostream>
#include <calculator.h>          // Angular brackets work via -I inc

int main() {
    Calculator calc;
    std::cout << "5 + 3 = " << calc.add(5, 3) << std::endl;
    std::cout << "10 - 4 = " << calc.subtract(10, 4) << std::endl;
    return 0;
}
```

4. Makefile

```
# Compiler and flags
CXX = g++
CXXFLAGS = -Wall -Wextra -g -I inc
LDFLAGS =

# Directories
SRC_DIR = src
INC_DIR = inc
OBJ_DIR = obj
BIN_DIR = bin

# Target
TARGET = $(BIN_DIR)/calculator.exe

# Source and object files
SOURCES = $(wildcard $(SRC_DIR)/*.cpp)
OBJECTS = $(patsubst $(SRC_DIR)/%.cpp,$(OBJ_DIR)/%.o,$(SOURCES))

# Default target
all: $(TARGET)

# Link object files
$(TARGET): $(OBJECTS) | $(BIN_DIR)
    $(CXX) $(LDFLAGS) $^ -o $@

# Compile source files
$(OBJ_DIR)/%.o: $(SRC_DIR)/%.cpp | $(OBJ_DIR)
    $(CXX) $(CXXFLAGS) -c $< -o $@

# Create directories
$(BIN_DIR):
    mkdir -p $(BIN_DIR)

$(OBJ_DIR):
    mkdir -p $(OBJ_DIR)

# Clean
clean:
    rm -rf $(OBJ_DIR)/*.o $(BIN_DIR)/*.exe

# Run
```

```
run: $(TARGET)
    ./$(TARGET)
```

```
.PHONY: all clean run
```

VS Code Configuration - Complete Guide

All configuration files go in the `.vscode/` folder inside your project.

1. tasks.json - Build Tasks Configuration

This file tells VS Code **how to compile/build** your project.

```

{
  "version": "2.0.0",          // tasks.json format version (use 2.0.0)
  "tasks": [                  // Array of build tasks
    {
      // — TASK 1: BUILD —————
      "label": "Build with Make", // Name shown in VS Code task list
      "type": "shell",           // Run as shell command
      "command": "C:/MinGW/bin/make.exe", // Path to make.exe
      "args": ["all"],           // Arguments: make all
      "group": {
        "kind": "build",         // Group type: "build" | "test" | "none"
        "isDefault": true       // This runs when you press Ctrl+Shift+B
      },
      "problemMatcher": ["$gcc"], // Parse g++ errors/warnings
      "detail": "Build project using Makefile" // Description
    },

    // — TASK 2: CLEAN —————
    {
      "label": "Clean Build",     // Task name
      "type": "shell",           // Run as shell command
      "command": "C:/MinGW/bin/make.exe", // Path to make.exe
      "args": ["clean"],         // Arguments: make clean
      "group": "build",          // Part of build group
      "problemMatcher": []       // No error parsing needed
    },

    // — TASK 3: RUN —————
    {
      "label": "Run Program",     // Task name
      "type": "shell",           // Run as shell command
      "command": "C:/MinGW/bin/make.exe", // Path to make.exe
      "args": ["run"],           // Arguments: make run
      "group": "test",           // Part of test group
      "problemMatcher": []       // No error parsing needed
    }
  ]
}

```

tasks.json Field Reference

Field	Values	Description
version	"2.0.0"	Format version, always use 2.0.0
label	Any string	Display name in VS Code task list
type	"shell"	Command type (shell or process)
command	Path/string	The executable to run
args	Array of strings	Command-line arguments
group.kind	"build" , "test" , "none"	Task category
group.isDefault	true / false	Run on Ctrl+Shift+B
problemMatcher	["\$gcc"]	Parse compiler errors (clickable in Problems panel)
detail	Any string	Optional description

Adding More Tasks

```
{  
  "label": "Run without rebuild",  
  "type": "shell",  
  "command": "./bin/calculator.exe",  
  "group": "test",  
  "problemMatcher": []  
}
```

2. launch.json - Debug Configuration

This file tells VS Code **how to debug** your program.


```

{
  "version": "0.2.0",           // launch.json format version (use 0.2.0)
  "configurations": [          // Array of debug configurations
    {
      // ——— DEBUGGER SETTINGS ———
      "name": "Launch Calculator", // Name shown in debug dropdown

      "type": "cppvsdbg",         // Debugger type:
                                   //   "cppvsdbg" = Visual Studio debugger (Windows)
                                   //   "cppdbg"   = GDB/MI debugger (MinGW)
                                   //   "lldb"    = LLDB debugger (macOS/Linux)

      "request": "launch",        // "launch" = start new process
                                   // "attach" = attach to running process

      // ——— PROGRAM SETTINGS ———
      "program": "${workspaceFolder}/bin/calculator.exe", // Path to .exe
                                                         // ${workspaceFolder} = project root folder

      "args": [],                 // Command-line args passed to program
                                   // Example: ["--verbose", "--file=test.txt"]

      "cwd": "${workspaceFolder}", // Working directory when running

      "environment": [],          // Environment variables
                                   // Example: [{"name": "PATH", "value": "/usr/bin"}]

      // ——— CONSOLE SETTINGS ———
      "console": "externalTerminal", // Where to show output:
                                       //   "externalTerminal" = new cmd window
                                       //   "integratedTerminal" = VS Code terminal
                                       //   "internalConsole" = Debug Console (no input)

      // ——— BEHAVIOR ———
      "stopAtEntry": false,        // true = stop at main() automatically
                                   // false = run normally until breakpoint

      // ——— BUILD BEFORE DEBUG ———
      "preLaunchTask": "Build with Make" // Run this task before debugging
                                           // Must match a task label in tasks.json
    }
  ]
}

```

```
]
}
```

launch.json Field Reference

Field	Values	Description
version	"0.2.0"	Format version, always use 0.2.0
name	Any string	Configuration name in debug dropdown
type	"cppvsdbg" , "cppdbg" , "lldb"	Debugger type
request	"launch" , "attach"	Start new or attach to running
program	Path to .exe	Executable to debug
args	Array of strings	Program arguments
cwd	Path	Working directory
environment	Array of objects	Environment variables
console	"externalTerminal" , "integratedTerminal" , "internalConsole"	Output location
stopAtEntry	true / false	Stop at main()
preLaunchTask	Task label string	Task to run before debug

Multiple Configurations

```
{
  "configurations": [
    {
      "name": "Debug (External Console)",
      "type": "cppvsdbg",
      "request": "launch",
      "program": "${workspaceFolder}/bin/calculator.exe",
      "console": "externalTerminal",
      "preLaunchTask": "Build with Make"
    },
    {
      "name": "Debug (Integrated Terminal)",
      "type": "cppvsdbg",
      "request": "launch",
      "program": "${workspaceFolder}/bin/calculator.exe",
      "console": "integratedTerminal",
      "preLaunchTask": "Build with Make"
    }
  ]
}
```

3. c_cpp_properties.json - IntelliSense Configuration

This file tells VS Code's **code completion and error checking** where to find headers.

```
{  
    "configurations": [ // Array of configurations  
        {  
            // — CONFIGURATION NAME —————  
            "name": "MinGW", // Name shown in bottom-right of VS Code  
                                // Click to switch configurations  
  
            // — INCLUDE PATHS —————  
            "includePath": [  
                "${workspaceFolder}/**", // Search all folders recursively  
                "${workspaceFolder}/inc" // Your custom headers folder  
            ],  
            // This allows: #include <calculator.h> to find inc/calculator.h  
  
            // — PREDEFINED MACROS —————  
            "defines": [  
                "_DEBUG", // Defined in debug builds  
                "UNICODE", // Unicode character support  
                "_UNICODE" // Unicode character support  
            ],  
            // Equivalent to: g++ -D_DEBUG -DUNICODE -D_UNICODE  
  
            // — COMPILER PATH —————  
            "compilerPath": "C:/MinGW/bin/g++.exe", // Path to g++  
            // Used by IntelliSense to determine:  
            //   - System include paths  
            //   - Default compiler flags  
            //   - C/C++ standard version  
  
            // — LANGUAGE STANDARD —————  
            "cStandard": "c11", // C standard: c89, c99, c11, c17  
            "cppStandard": "c++17", // C++ standard: c++98, c++11, c++14, c++17, c++20  
  
            // — INTELLISENSE MODE —————  
            "intelliSenseMode": "windows-gcc-x64" // Mode for Windows + GCC:  
                                                    // "windows-gcc-x64" = 64-bit MinGW  
                                                    // "windows-gcc-x86" = 32-bit MinGW  
                                                    // "windows-msvc-x64" = Visual Studio  
        }  
    ],  
    "version": 4 // Config file version (use 4)  
}
```

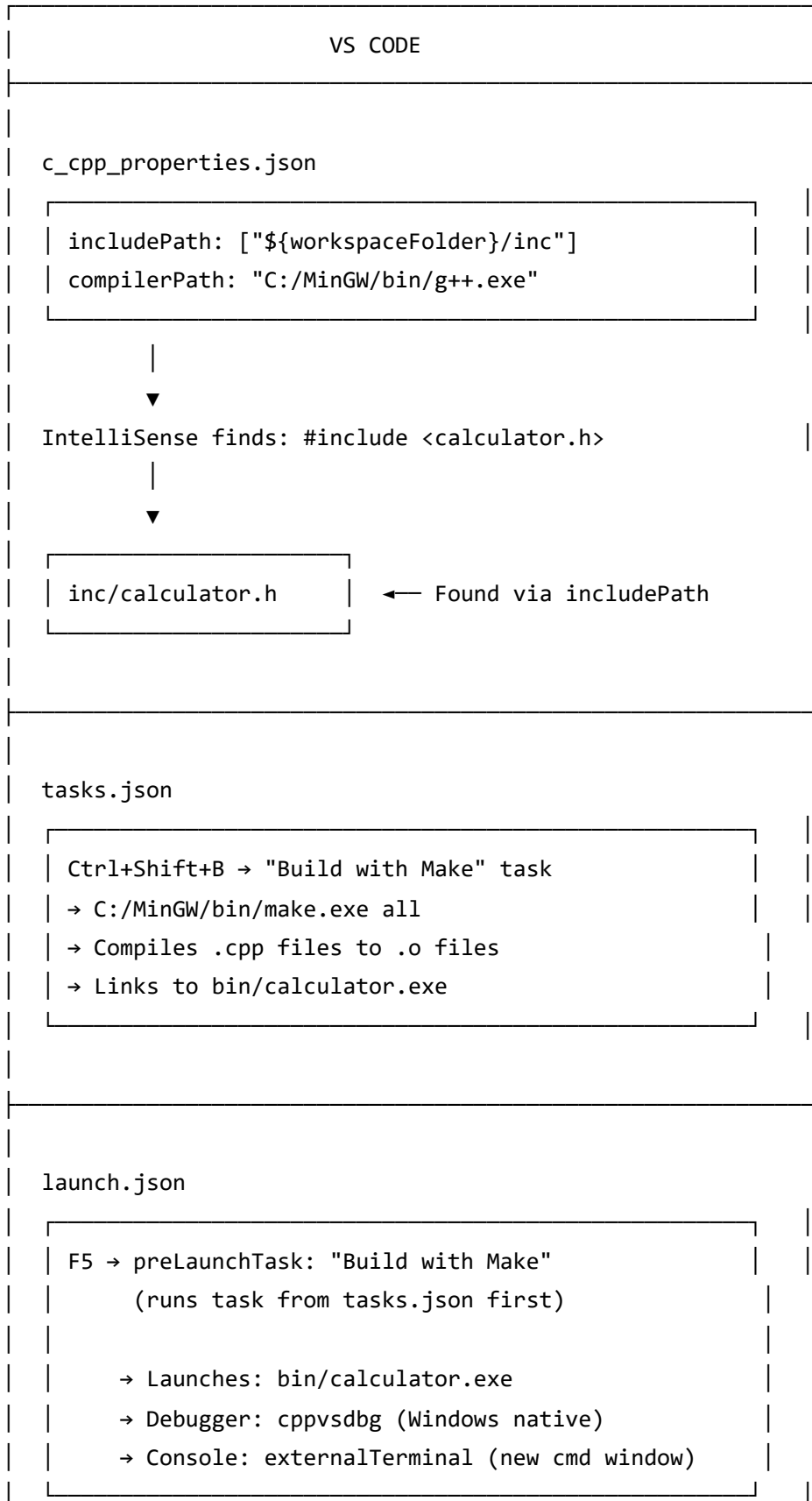
c_cpp_properties.json Field Reference

Field	Values	Description
version	4	Config format version
name	Any string	Configuration name
includePath	Array of paths	Where to find header files
defines	Array of strings	Preprocessor macros
compilerPath	Path to g++/clang++	Used for system headers
cStandard	"c89" , "c99" , "c11" , "c17"	C language standard
cppStandard	"c++98" to "c++20"	C++ language standard
intelliSenseMode	See below	Platform + compiler mode

IntelliSense Modes

Mode	Platform	Compiler
windows-gcc-x64	Windows	MinGW 64-bit
windows-gcc-x86	Windows	MinGW 32-bit
windows-msvc-x64	Windows	Visual Studio
windows-clang-x64	Windows	Clang
linux-gcc-x64	Linux	GCC
macos-clang-x64	macOS	Clang

4. How It All Connects



5. Customization Examples

Change compiler path (different MinGW location)

```
// In tasks.json:  
"command": "C:/my-libs/mingw/bin/make.exe"  
  
// In c_cpp_properties.json:  
"compilerPath": "C:/my-libs/mingw/bin/g++.exe"
```

Change include path (multiple folders)

```
"includePath": [  
    "${workspaceFolder}/**",  
    "${workspaceFolder}/inc",  
    "${workspaceFolder}/third_party/include"  
]
```

Enable debug console input (for interactive programs)

```
// In launch.json:  
"console": "integratedTerminal" // Use VS Code terminal (can type input)
```

Pass arguments to program

```
// In launch.json:  
"args": ["--input", "data.txt", "--verbose"]
```

Stop at main() for debugging

```
// In launch.json:  
"stopAtEntry": true // Debugger pauses at first line of main()
```

Build & Run

Method 1: VS Code

Action	Shortcut
Build	Ctrl+Shift+B
Run	F5

Method 2: Terminal

```
# Build
C:/MinGW/bin/make.exe all

# Run
./bin/calculator.exe

# Clean
C:/MinGW/bin/make.exe clean
```

Debug

1. Open a .cpp file
2. Click left of line number to set breakpoint (red dot)
3. Press F5 to start debugging

Debug Controls

Button	Action
 Continue	Run until next breakpoint
 Step Over	Execute next line
 Step Into	Enter function
 Step Out	Exit current function

Button	Action
 Restart	Restart debugging
 Stop	Stop debugging

Troubleshooting

Issue: "g++ not found"

Fix: Add MinGW to PATH:

```
# Temporary (current session)
$env:Path += ";C:\MinGW\bin"

# Permanent
[System.Environment]::SetEnvironmentVariable("Path", $env:Path + ";C:\MinGW\bin", "User")
```

Issue: "Unable to start debug"

Fix: Use `cppvsdbg` instead of `cppdbg` in `launch.json`:

```
"type": "cppvsdbg"
```

Issue: Custom headers not found

Fix: Ensure `c_cpp_properties.json` has:

```
"includePath": ["${workspaceFolder}/inc"]
```

And Makefile has `-I inc` flag.

Issue: Makefile tabs error

Fix: Makefile requires **tabs**, not spaces for indentation.

- Open Makefile in VS Code

- Enable: View → Render Whitespace
- Ensure recipe lines start with Tab character

Issue: Red squiggly lines under headers

Fix: Press `Ctrl+Shift+P` → `C/C++: Reset IntelliSense Database`

Quick Reference

Task	Command
Build	<code>Ctrl+Shift+B</code>
Debug	<code>F5</code>
Clean	Terminal: <code>make clean</code>
Add new .cpp	Create in <code>src/</code> , rebuild
Add new .h	Create in <code>inc/</code> , include with <code><></code>

Adding New Files

1. Create header in `inc/newfile.h`
2. Create source in `src/newfile.cpp`
3. Include header using angular brackets:

```
#include <newfile.h>
```

4. Press `Ctrl+Shift+B` to rebuild

Makefile automatically detects new `.cpp` files in `src/` .

Guide created for CppCalculator project